

Fase 3 - Estructuras repetitivas y arreglo

Cristian Fernando Solano Villamizar

Marzo 2026.

Universidad Nacional abierta y a distancia-UNAD

Escuela de Ciencias Básicas Tecnología e Ingeniería

Fundamentos de Programación

213022_702

Introducción

Este documento describe la solución a dos problemas de programación lógica creados con el lenguaje Python, los cuales tienen como objetivo la administración de datos y la automatización de procesos. En el primer escenario, se utilizan estructuras de repetición para consolidar datos estadísticos de los aficionados al fútbol, lo que posibilita calcular frecuencias y promedios de preferencias. En el segundo caso, se emplean matrices para manejar las ventas de una pizzería, mejorando la consulta de datos relacionados con los precios y las porciones mediante índices. El propósito fundamental es evidenciar la eficacia de las estructuras de almacenamiento y los ciclos para solucionar dificultades reales, asegurando la integridad de la información a través del uso de métodos de gestión de excepciones y validación de datos.

Problema 5: Estructuras repetitivas

Una nueva tienda deportiva desea saber qué productos ofrecer para los hinchas de los equipos de fútbol colombianos. Para esto, se quiere preguntar a 10 fanáticos del fútbol:

- Cuál es su equipo favorito.
- Qué tipo de artículo deportivo prefiere entre camiseta, chaqueta, gorra o morral con el logo de su equipo favorito.
- Edad del fanático del fútbol.
- Cantidad de días al año en los cuales asisten al estadio a partidos de su equipo favorito.

El programa debe mostrar:

- Cantidad de fanáticos por equipo de fútbol
- Promedio de edades de los hinchas por cada equipo de fútbol.
- El tipo de artículo deportivo preferido por los hinchas
- Promedio de días al año en los cuales los hinchas asisten al Estadio

```
#-----  
print("<----- TIENDA DEPORTIVA ----->")  
print("<----- REGISTRO DE HINCHAS FÚTBOL ----->")  
print("<--- ARTÍCULOS: CAMISETA, CHAQUETA, GORRA, MORRAL --->")  
  
# 1 - VARIABLES GLOBALES PARA ACUMULAR DATOS  
total_hinchas = 10  
asistencia_total = 0  
datos Equipos = {} # Diccionario para guardar nombre del equipo y lista  
de edades  
conteo_articulos = {"camiseta": 0, "chaqueta": 0, "gorra": 0, "morral":  
0}
```

```
# 2 - ESTRUCTURA REPETITIVA (PARA 10 FANÁTICOS)

for i in range(1, total_hinchas + 1):
    print(f"\n<----- REGISTRO FANÁTICO No. {i} ----->")

    # VALIDACIÓN DEL EQUIPO .strip() quita espacios y .capitalize()
    asegura que "equipo" y "EQUIPO" sean "Equipo"
    equipo = input("<--- Ingrese su equipo favorito:
").strip().capitalize()

    # VALIDACIÓN DEL ARTÍCULO (Solo permite los 4 definidos)
    while True:
        articulo = input("<--- Artículo preferido
(Camiseta/Chaqueta/Gorra/Morral): ").lower().strip()
        if articulo in conteo_articulos:
            conteo_articulos[articulo] += 1
            break
        else:
            print("<--- [ERROR]: Artículo no válido. Intente de nuevo.")

    # VALIDACIÓN DE EDAD (Evita que el programa falle si ingresan letras)
    while True:
        try:
            edad = int(input("<--- Ingrese su edad: "))
            if edad < 0:
                print("<--- [ERROR]: La edad no puede ser negativa.")
                continue
            break
        except ValueError:
            print("<--- [ERROR]: Por favor, ingrese un número entero para
la edad.")

    # VALIDACIÓN DE ASISTENCIA
    while True:
        try:
            asistencia = int(input("<--- Días que asiste al estadio al
año: "))
            if asistencia < 0:
                print("<--- [ERROR]: Los días no pueden ser negativos.")
                continue
            break
        except ValueError:
            print("<--- [ERROR]: Por favor, ingrese un número para los
días.")

    # PROCESAMIENTO DENTRO DEL CICLO

    # Acumular asistencia para el promedio general
```

```
asistencia_total += asistencia

# Organizar edades por equipo
if equipo not in datos_equipos:
    datos_equipos[equipo] = []
datos_equipos[equipo].append(edad)

# 3 - MOSTRAR RESULTADOS Y ESTADÍSTICAS

print("\n<----- RESUMEN DE DATOS ----->")

# A. CANTIDAD Y PROMEDIO DE EDAD POR EQUIPO
print("<--- Estadísticas por Equipo:")
for equipo, edades in datos_equipos.items():
    cantidad = len(edades)
    promedio_edad = sum(edades) / cantidad
    print(f"<--- {equipo}: {cantidad} Hinchas | Edad Promedio:
{promedio_edad:.1f}")

# B. ARTÍCULO PREFERIDO
articulo_ganador = max(conteo_articulos, key=conteo_articulos.get)
votos_articulo = conteo_articulos[articulo_ganador]
print("<--- Artículo más pedido:", articulo_ganador.capitalize(),
f"({votos_articulo} hinchas)")

# C. PROMEDIO DE ASISTENCIA AL ESTADIO
promedio_asistencia = asistencia_total / total_hinchas
print(f"<--- Promedio de asistencia general: {promedio_asistencia:.1f}
días/año")

# 4 - MENSAJE DE DESPEDIDA

print("<----->")
print("<----- GRACIAS POR USAR EL SISTEMA ----->")
print("<----- REALIZADO POR CRISTIAN SOLANO ----->")
print("<----- GRUPO NUMERO 213022_302 ----->")
```

Tabla de requerimientos

Identificación del requerimiento	Descripción	Entradas	Resultados o salidas
R1	Captura de datos del hincha: El sistema debe permitir el ingreso de equipo, artículo, edad y días de asistencia.	Nombre del equipo, tipo de artículo, edad (entero), días (entero).	Datos validados y almacenados temporalmente en el ciclo.
R2	Validación de tipos de datos: El sistema debe impedir el avance si se ingresan letras en campos numéricos (Edad/Días).	Input (Entradas de usuario).	Mensajes de error ("Ingrese un número entero").
R3	Unificación de nombres de equipos: El sistema debe tratar igual los nombres sin importar mayúsculas o minúsculas.	Texto del usuario (ej: "equipo", "EQUIPO").	Almacenamiento unificado (ej: "Equipo").
R4	Cálculo de estadísticas por equipo: Contabilizar	Lista de edades	Cantidad de hinchas y

	hinchas y promediar edades por cada club registrado.	agrupadas por equipo.	promedio de edad por equipo.
R5	Determinación de preferencia de artículos: Identificar cuál de los 4 artículos disponibles tuvo más votos.	Conteo de (Camiseta, Chaqueta, Gorra, Morral).	Nombre del artículo más pedido y su cantidad.
R6	Promedio general de asistencia: Calcular la media de días que los 10 hinchas asisten al estadio.	Sumatoria de días de asistencia.	Valor numérico del promedio general.

Problema 6: Arreglos (matrices)

Una pizzería desea desarrollar una aplicación para automatizar el proceso de ventas en el año 2026. La pizzería ofrece 5 tipos de pizza, cada una con un tamaño y precio específico, según la siguiente información

Tipo	Tamaño	Porciones	Precio
1	Pequeña	(2 porciones)	\$10.000
2	Mediana	(4 porciones)	\$15.000
3	Larga	(6 porciones)	\$20.000
4	Familiar	(8 porciones)	\$25.000
5	Extra Familiar	(12 porciones)	\$35.000

Requisitos del programa

1. El sistema debe permitir atender a un número X de clientes.
2. Cada cliente puede comprar varios tipos de pizza en una misma orden. - Por ejemplo: un cliente elige Pequeña (tipo 1), Larga (tipo 3) y Familiar (tipo 4). El precio total será \$55.000 y el número total de porciones será 16.
3. **Al finalizar todas las ventas, el programa debe mostrar:**
 - Cantidad total de pizzas vendidas por tipo.
 - Valor total de las ventas.
 - Número total de porciones vendidas.

Al finalizar, el sistema debe mostrar los siguientes resultados:

- Cantidad de pizzas vendidas por tipo.
- Precio total de las pizzas vendidas.
- El número total de porciones vendidas.

La pizzería usa una base de datos basada en Arreglos de la siguiente forma:

Arreglo con el precio de las pizzas:

- posición 1 representa la pizza Pequeña (tipo 1) y tiene el valor de 10000
- posición 2 representa la pizza Mediana (tipo 2) y tiene el valor de 15000
- posición 3 representa la pizza Larga (tipo 3) y tiene el valor de 20000
- posición 4 representa la pizza Familiar (tipo 4) y tiene el valor de 25000
- posición 5 representa la pizza Extra Familiar (tipo 5) y tiene el valor de 35000

Arreglo con el número de porciones de las pizzas:

- posición 1 representa la pizza Pequeña (tipo 1) y tiene el valor de 2 porciones

- posición 2 representa la pizza Mediana (tipo 2) y tiene el valor de 4 porciones
- posición 3 representa la pizza Larga (tipo 3) y tiene el valor de 6 porciones
- posición 4 representa la pizza Familiar (tipo 4) y tiene el valor de 8 porciones
- posición 5 representa la pizza Extra Familiar (tipo 5) y tiene el valor de 12 porciones

Tabla de requerimientos

Identificación del requerimiento	Descripción	Entradas	Resultados o salidas
R1	Definir base de datos de productos	Arreglos de constantes de precios y porciones	Estructura de datos lista
R2	Gestionar múltiples clientes	Cantidad de clientes (X)	Ciclo principal de atención

R3	Permitir múltiples pizzas por orden	Tipo de pizza (1-5)	Total acumulado por cliente
R4	Generar reporte consolidado	Datos acumulados en los arreglos	Ventas totales, porciones y conteo por tipo

Código fuente:

```

print("<----- PIZZERÍA ---->")
print("<----- SISTEMA DE VENTAS AUTOMATIZADO ----->")

# 1 - BASE DE DATOS BASADA EN ARREGLOS O LISTAS
# Usamos el índice 0 como vacío para que la posición 1 coincida con el
Tipo 1

precios = [0, 10000, 15000, 20000, 25000, 35000]
porciones = [0, 2, 4, 6, 8, 12]
nombres = ["", "Pequeña", "Mediana", "Larga", "Familiar", "Extra
Familiar"]

# VARIABLES PARA ACUMULAR TOTALES GENERALES
cantidades_por_tipo = [0, 0, 0, 0, 0, 0] # Arreglo para contar cuántas de
cada una
valor_total_ventas = 0
total_porciones_vendidas = 0

# 2 - INGRESO DE CLIENTES
while True:
    try:
        numero_clientes = int(input("<--- ¿Cuántos clientes desea
atender?: "))
        if numero_clientes > 0: break
        print("<--- [ERROR]: Debe ser al menos 1 cliente.")
    except ValueError:
        print("<--- [ERROR]: Ingrese un número válido.")

# 3 - ESTRUCTURA REPETITIVA PARA CLIENTES
for c in range(1, numero_clientes + 1):
    print(f"\n<----- ATENDIENDO CLIENTE No. {c} ----->")
    total_cliente = 0
  
```

```
porciones_cliente = 0

while True:
    print("\n<--- MENÚ DE PIZZAS >")
    print("1. Pequeña ($10.000)  2. Mediana ($15.000)  3. Larga ($20.000)")
    print("4. Familiar ($25.000)  5. Extra Familiar ($35.000)")

    try:
        tipo = int(input("<--- Elija el tipo de pizza (1-5) o 0 para finalizar orden: "))

        if tipo == 0:
            break
        elif 1 <= tipo <= 5:
            # PROCESAMIENTO USANDO LOS ARREGLOS
            total_cliente += precios[tipo]
            porciones_cliente += porciones[tipo]

            # ACUMULADORES GLOBALES
            cantidades_por_tipo[tipo] += 1
            valor_total_ventas += precios[tipo]
            total_porciones_vendidas += porciones[tipo]

            print(f" [OK] Añadida: Pizza {nombres[tipo]}")
        else:
            print("<--- [ERROR]: Tipo de pizza no válido.")
    except ValueError:
        print("<--- [ERROR]: Ingrese solo números (1-5).")

    print(f"<--- TOTAL ORDEN CLIENTE {c}: ${total_cliente}")
    print(f"<--- TOTAL PORCIONES CLIENTE {c}: {porciones_cliente}")

# 4 - MOSTRAR RESULTADOS FINALES
print("\n<----- REPORTE DE CIERRE ----->")
print("<--- Cantidad de pizzas vendidas por tipo:")
for i in range(1, 6):
    print(f"<--- Tipo {i} ({nombres[i]}): {cantidades_por_tipo[i]} unidades")

print("<----->")
print(f"<--- VALOR TOTAL DE VENTAS: ${valor_total_ventas}")
print(f"<--- TOTAL PORCIONES VENDIDAS: {total_porciones_vendidas}")

# 5 - PIE DE PÁGINA
print("<----->")
print("<----- REALIZADO POR CRISTIAN SOLANO ----->")
print("<----- GRUPO NUMERO 213022_302 ----->")
```

Conclusiones

- Se concluyó que las estructuras que se repiten (for y while) son fundamentales para manejar grandes cantidades de datos eficientemente, disminuir la redundancia en el código y simplificar el control de flujo en aplicaciones interactivas.
- El empleo de arreglos (listas) posibilitó la estructuración lógica de la base de datos de productos, lo que evidenció que enlazar datos por medio de índices es una estrategia más eficiente que manejar múltiples variables independientes para el control de precios e inventarios.
- Para desarrollar un software sólido, fue esencial implementar ciclos de validación y bloques try-except, garantizando que el programa pueda manejar los errores de entrada del usuario sin detener la ejecución del sistema.

Bibliografía

- ☐ Guido van Rossum. (2026). *Python Documentation: Data Structures*. Python Software Foundation.
<https://docs.python.org/3/tutorial/datastructures.html>
- ☐ Joyanes Aguilar, L. (2020). *Fundamentos de programación: Algoritmos, estructura de datos y objetos* (5ta ed.). McGraw-Hill.
- ☐ Universidad Nacional Abierta y a Distancia. (2026). *Unidad 2: Estructuras de control y arreglos en Python*. [Material de estudio]. UNAD.
- ☐ Moyano, Ricardo. (2023). *Manejo de excepciones en Python: Guía definitiva*. Real Python. <https://realpython.com/python-exceptions/>